数学与 CS 问答笔记

math-answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 03, 2026

Contents

1. 给 go doc 配上一组 fish 函数（bat 高亮 + godochelp 自述） 3
   1.1. 费曼直觉 3
   1.2. 硬核机制 3
      1.2.1. 放在哪：fish 的 functions 目录 3
      1.2.2. 三个函数 4
      1.2.3. 实测：函数被识别，且立即可用 4
      1.2.4. 第四个：godochelp（家族自述） 5
   1.3. 陷阱与易错 6
   1.4. 关键点 7
   1.5. 变式检验 7
   1.6. 同类归类 7
   1.7. Go 语言机制补充 7
      1.7.1. go doc 的三档粒度与对应入口 7
   1.8. 本次操作与当前状态 8
      1.8.1. 做过的操作 8
      1.8.2. 当前状态 8
      1.8.3. 待办 8
Index of Listings 9

1. 给 go doc 配上一组 fish 函数（bat 高亮 + godochelp 自述）

1.1. 费曼直觉

`go doc` 的输出是纯文本，在终端里既没有语法高亮、也没有分页。包一层 `bat` 就补上这两点，而包成函数的意义在于**把三种查询粒度变成三个肌肉记忆**：

1. `godocbat` —— 平时用得最多：查包、查 API、查类型、查方法；
2. `godocall` —— 扫整个包，建立全局观；
3. `godocsrc` —— 看实现源码。

三个函数的实现都只有一行，真正的价值在于“顺手”。

1.2. 硬核机制

1.2.1. 放在哪：fish 的 functions 目录

Fish 有两条自动加载路径，惯例上分得很清：

位置	适合放什么	加载时机
`~/.config/fish/functions/<名字>.fish`	独立函数，**一个函数一个文件**，文件名 = 函数名	**用到时才加载**（autoload）
`~/.config/fish/conf.d/<任意名>.fish`	环境变量、PATH、别名之类的启动配置	每次开 shell 都 source

本机的实际情况：`~/.config/fish` 是 dotfiles 仓库的 symlink（指向 `~/dotfiles/fish/.config/fish`），而 `functions/` 里原本已有 `extract.fish`、`update-reasonix.fish`——新函数就按同样的惯例放进去。

> 📌 **核心要点** 放 `functions/` 而不是塞进 `config.fish` 的原因就在“加载时机”那一列：**autoload 让你不必为用不到的函数付出启动开销**。fish 在启动时不读这些文件，直到第一次输入 `godocbat` 才去加载对应文件——所以新加函数**不需要重启 shell**。

1.2.2. 三个函数

```
# ~/.config/fish/functions/godocbat.fish
function godocbat --description '查看 Go package 或 API (bat 高亮) '
    go doc $argv | bat --language=go
end

# godocall.fish
function godocall --description '查看 Go package 完整文档 (bat 高亮) '
    go doc -all $argv | bat --language=go
end

# godocsrc.fish
function godocsrc --description '查看 Go API 的源码实现 (bat 高亮) '
    go doc -src $argv | bat --language=go
end
```

Listing 1: 三个函数：只是把 go doc 的输出接给 bat，参数原样透传

每个文件顶部还留了用法注释块（与 `update-reasonix.fish` 的既有风格一致），例如 `godocbat.fish`：

```
# godocbat.fish — 查看 Go package 或具体 API，并用 bat 高亮阅读
#
# 用法:
#   godocbat slices                       # 查看 slices 包概览
#   godocbat slices.Delete                # 查看某个 API
#   godocbat sync.WaitGroup               # 查看具体类型（含方法清单）
#   godocbat strings.Builder.WriteString  # 查看方法
#
# 说明: 等价于 go doc <参数>，只是把输出交给 bat 上色。参数规则与 go doc 完全一致——
#       不支持 ./... 这类模式；查自己的包要用 ./internal/db 这种相对路径。
```

Listing 2: 文件顶部的用法注释：`functions -v` 看不到它，但打开文件就能看到

1.2.3. 实测：函数被识别，且立即可用

```
$ fish -c 'functions -v godocbat'
# 定义于 /home/zz/.config/fish/functions/godocbat.fish @ line 11
function godocbat --description '查看 Go package 或 API (bat 高亮) '
    go doc $argv | bat --language=go
end
```

Listing 3: fish 自动从 functions 目录找到定义，并被 –description 标记出来

实际调用（输出经 bat 上色，此处为纯文本）：

```
$ godocbat slices.Delete
package slices // import "slices"

func Delete[S ~[]E, E any](s S, i, j int) S
    Delete modifies s in place by removing the elements s[i:j], and returns the
    modified slice. Delete panics if j > len(s) or s[i:j] is not a valid slice
    of s. Delete is O(len(s)-i), so if many items must be deleted, it is better
    to make a single call deleting them all together than to delete one at a
    time.

$ godocsrc slices.Delete
func Delete[S ~[]E, E any](s S, i, j int) S {
    …（函数体）
```

Listing 4: godocbat 给用法，godocsrc 给实现——两个函数正好覆盖 go doc 的两档粒度

1.2.4. 第四个：godochelp（家族自述）

函数一多，最需要的是“别再问我有什么”——所以补一个自述函数。它不查文档，只把家族成员、用法示例、与原生命令的对应关系打印出来，并用 `set_color` 做了简单配色：

```
function godochelp --description '显示 godoc 函数家族的使用方法'
    set_color brblue
    echo 'godoc 函数家族 — 用 bat 高亮阅读 Go 文档（离线，与当前 Go 版本严格对应）'
    set_color normal
    echo
    set_color bryellow
    echo '  成员'
    set_color normal
    echo '    godocbat   <包|符号>   查看 package 或 API 的用法（含类型的方法清单）'
    echo '    godocall   <包>        查看整个 package 的完整文档'
    echo '    godocsrc   <符号>      查看指定 API 的源码实现'
    echo '    godochelp              显示这份帮助'
    #  （其余三段：用法示例 / 原生命令对应 / 提示，完整实现见 dotfiles 仓库）
end
```

Listing 5: godochelp 用 set_color 分段着色；它不经过 bat，因为要自己控制颜色

实际输出：

```
$ godochelp
godoc 函数家族 — 用 bat 高亮阅读 Go 文档（离线，与当前 Go 版本严格对应）

  成员
    godocbat   <包|符号>   查看 package 或 API 的用法（含类型的方法清单）
    godocall   <包>        查看整个 package 的完整文档
    godocsrc   <符号>      查看指定 API 的源码实现
    godochelp              显示这份帮助

  用法示例
    godocbat slices                        # 包概览
    godocbat slices.Delete                 # 某个 API
    godocbat sync.WaitGroup                # 具体类型（含方法清单）
    godocbat strings.Builder.WriteString   # 方法
    godocbat ./internal/db.KV              # 自己的包（相对路径）
    godocall slices                        # 整个包的完整文档
    godocsrc slices.Delete                 # 源码实现

  与原生命令的对应
    godocbat  ->  go doc <参数>
    godocall  ->  go doc -all <包>
    godocsrc  ->  go doc -src <符号>

  提示
    · 参数规则与 go doc 一致：不支持 ./... 这类模式
    · 查自己的包用相对路径，例如 ./internal/db 或 ./internal/db.KV
    · 看某个函数定义在哪：functions -v godocbat
    · 原生帮助：go help doc
```

Listing 6: godochelp 的输出：成员、示例、对应关系、提示四段

> 💡 **背景与类比** 这一层“自述函数”的惯例在 CLI 工具里很常见（ `kubectl help` 、 `gh help` 、 `cargo --help` ）：**让工具自己说明自己**，比在 wiki 里找文档更快。它同时也是备忘——几个月后回到这套函数，敲一下就知道每个命令是干什么的。

1.3. 陷阱与易错

1. **把函数写进 `config.fish`** 。能工作，但每个 shell 启动都要解析所有函数； `functions/` + autoload 才是 fish 的推荐做法；
2. **文件名与函数名不一致**。autoload 的查找规则是“按名字找同名文件”， `godocbat.fish` 里定义 `function godoc_bat` 就永远不会被自动加载；
3. **忘了 bat**，那么管道右端会报 `command not found` 。本机已装（ `bat 0.26.1` ），换机器时要留意；
4. **以为 `--language=go` 可有可无**。它保证 bat 用 Go 语法规则着色；不指定的话 bat 多数情况下也能猜对，但显式指定更稳。

1.4. 关键点

> 📌 **核心要点** fish 的函数放 `functions/`（一个函数一个文件、文件名同名、autoload 按需加载）；三个 `go doc` 包装分别对应**概览 / 完整文档 / 源码**三档粒度。新增函数后**不必重启 shell**，直接敲名字即可。

1.5. 变式检验

1. **换成给 `go doc` 加分页**：把 `bat` 换成 `bat --paging=always`（或直接 `less -R`），适合 `godocall` 这种长输出；
2. **换成查自己的包**：`godocbat ./internal/db.KV` —— 相对路径包同样透传，日常写库时很顺手；
3. **换成把常用查询做成补全**：fish 的 `complete -c godocbat -a "(go list std | string replace 'go/' '')"` 可以让包名补全，属于进一步优化。

1.6. 同类归类

1. 与 `go help`、`man`、`tldr` 的关系：这条管道本质是“给命令行文档加一层阅读器”，和 `man bat`、`tldr go` 属于同一类工具习惯；
2. 与 shell 函数设计的对照：三个函数都保持“参数原样透传”（`$argv` 不加工），这样它们永远不会与 `go doc` 自身的行为分叉——**薄的包装优于聪明的包装**；
3. 往下延伸：课程后期会频繁查标准库（`encoding/binary`、`sort`、`os`），这三个函数正好覆盖“先看用法、再看实现”的日常节奏。

1.7. Go 语言机制补充

1.7.1. go doc 的三档粒度与对应入口

粒度	函数	命令
包概览	`godocbat strings`	`go doc strings`（67 行：包注释 + 符号摘要）
包完整文档	`godocall strings`	`go doc -all strings`（427 行：每个符号的完整说明）
单符号用法	`godocbat strings.Builder.WriteString`	`go doc <包>.<类型>.<方法>`
单符号实现	`godocsrc strings.Builder.WriteString`	`go doc -src <符号>`（含函数体）

> 📌 **核心要点** 一句话收束：**三个函数只是 `go doc` 的三档视图 + bat 的着色。**日常顺序建议：`godocbat <包>` 探路 → `godocall <包>` 建全局观 → `godocsrc <符号>` 看实现，需要追到底时再去 `/usr/lib/go/src/` 翻全文。

1.8. 本次操作与当前状态

1.8.1. 做过的操作

操作	细节
先摸清惯例	`~/.config/fish` 是 `~/dotfiles/fish/.config/fish` 的 symlink；`functions/` 下已有 `extract.fish`、`update-reasonix.fish`；`conf.d/` 里只有 PATH 等配置、没有函数定义
新建三个函数文件	`godocbat.fish`、`godocall.fish`、`godocsrc.fish`，风格对齐 `update-reasonix.fish`（顶部用法注释块 + `--description`）
实测验证	`fish -c 'functions -v godocbat'` 能列出定义；`godocbat slices.Delete`、`godocsrc slices.Delete` 均正常输出

1.8.2. 当前状态

三个函数已就位并可立即使用（fish autoload，无需重启 shell）：

```
godocbat slices                  # 包概览 / API / 类型 / 方法
godocall slices                  # 整个包的完整文档
godocsrc slices.Delete           # 源码实现

godocbat ./internal/db.KV        # 查本项目自己的包（相对路径）
```

Listing 7: 四个常用写法

1.8.3. 待办

1. 无（若要更进一步，可给 `godocbat` 加包名补全，见“变式检验”最后一条）。

Index of Listings

Listing 1 三个函数：只是把 go doc 的输出接给 bat，参数原样透传 4
Listing 2 文件顶部的用法注释：`functions -v` 看不到它，但打开文件就能看到 4
Listing 3 fish 自动从 functions 目录找到定义，并被 –description 标记出来 4
Listing 4 godocbat 给用法，godocsrc 给实现——两个函数正好覆盖 go doc 的两档粒度 . 5
Listing 5 godochelp 用 set_color 分段着色；它不经过 bat，因为要自己控制颜色 5
Listing 6 godochelp 的输出：成员、示例、对应关系、提示四段 6
Listing 7 四个常用写法 8